

OBJECT-ORIENTED ANALYSIS AND UML-BASED DESIGN OF AN AUTONOMOUS DRONE-BASED DISASTER RESPONSE AND RESCUE SYSTEM (ADDRRS)

1. Problem Analysis and Requirement Summary

Natural and man-made disasters such as floods, earthquakes, industrial fires and landslides often make affected areas difficult and dangerous for human rescue teams to enter immediately. In the critical first hours after such an event, the biggest challenge faced by emergency services is the absence of accurate, real-time information about where victims are located, how severe the damage is, and which routes remain safe for ground teams to use.

The Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) is conceived as a software platform that coordinates a fleet of camera- and sensor-equipped autonomous drones to survey disaster zones on behalf of human responders. Rather than sending rescue personnel into an unassessed area, the system allows an Emergency Control Officer to register an incident, mark out the affected zone on a map, and release one or more drones to autonomously fly a planned route, capture thermal, visual and gas-sensor data, and relay this information back to a central command server in near real time.

Whenever the on-board detection logic identifies a probable victim or a hazardous reading, the system is expected to automatically raise a prioritised alert and push a notification to the nearest rescue squad and to the officer supervising the mission, without waiting for someone to manually review the video feed. Once ground teams have acted on the alert, the mission is tracked through to completion and a structured report is produced summarising drone activity, detections, and outcomes. Because the platform must coordinate several independent kinds of behaviour — spatial data about zones and routes, autonomous device behaviour, human coordination, and asynchronous alerting — it is a natural candidate for Object-Oriented Analysis and Design (OOAD). Modelling the domain as a set of collaborating objects, each with a clearly bounded responsibility, makes it possible to reason about the system's structure and behaviour before a single line of code is written, and gives the resulting design the low coupling and high cohesion needed to extend it later (for example, adding a new sensor type or a new class of responder) without disturbing unrelated parts of the system.

2.Objectives:

The ADDRRS design is guided by the following objectives:

- Enable authorised emergency personnel to register disaster incidents and delineate affected zones quickly during a live event.
- Allow autonomous drones to be assigned to missions and to plan (and, if necessary, replan) flight routes over an affected zone.
- Continuously capture thermal, visual and environmental sensor data while a drone is airborne, and relay it to the command centre.
- Automatically flag probable victims or hazardous conditions from the captured data without requiring constant human monitoring.
- Raise prioritised alerts and push notifications to the appropriate rescue squad and supervising officer as soon as a detection occurs.
- Track the status of every drone and mission in real time, including battery state, faults, and route progress.
- Produce a structured mission-completion report once a zone has been fully surveyed or a mission is closed.
- Apply object-oriented design axioms and patterns so the resulting system is loosely coupled, highly cohesive, and easy to extend.

3. Scope of the System

ADDRRS is scoped as the software platform that sits between the field hardware (drones and their sensors) and the human command-and-control chain (emergency officers, drone pilots and rescue squads). The system is responsible for the following in-scope capabilities:

- Disaster incident registration and affected-zone definition.
- Drone fleet registration, availability tracking and mission assignment.
- Flight-route planning and dynamic route recalculation.
- Real-time drone telemetry monitoring (position, battery, health).
- Ingestion of camera and sensor data streamed from airborne drones.
- Automated victim and hazard detection from the ingested data.
- Alert generation and multi-channel notification dispatch.
- Mission-status tracking, acknowledgement handling and completion reporting.
- User account and role management for the different classes of operator.

The physical design of the drone airframe, the low-level flight-control firmware, and the training of any machine-learning model used for image classification are treated as external, pre-existing components that ADDRRS integrates with rather than builds; the software design instead concentrates on the coordination, data-management and decision-support responsibilities described above. Throughout, the design keeps real-time communication, autonomous operation, failure handling, security, reliability, scalability and maintainability as explicit cross-cutting concerns rather than afterthoughts.

4. Actor Identification

Reading the problem statement scenario by scenario surfaces five human/system actors that interact directly with ADDRRS, and one external system that the platform depends on but does not own:

Actor	Type	Responsibility in the System
Emergency Control Officer (ECO)	Primary – Human	Registers disaster incidents, defines affected zones, authorises mission assignment and reviews mission reports.
Drone Pilot / Operator	Primary – Human	Plans and adjusts flight routes, monitors live drone telemetry and takes manual control if required.
Rescue Squad Leader	Primary – Human	Receives and acknowledges alerts, coordinates the ground response and updates mission/victim status.
System Administrator	Primary – Human	Creates and manages user accounts, assigns roles and configures system-wide parameters.
Autonomous Drone	Primary – System actor	Executes flight routes, captures sensor/camera data, reports telemetry and executes autonomous safety behaviour (e.g., low-battery return).
Notification Gateway	Secondary – External system	Delivers SMS/push notifications generated by ADDRRS to the appropriate human actors' devices.

5. Functional Requirements

ID	Requirement	Description
FR-1	Incident Registration	The system shall allow an Emergency Control Officer to record a new disaster incident with its type, severity and location.
FR-2	Zone Definition	The system shall allow one or more affected zones to be drawn and associated with a registered incident.
FR-3	Drone Fleet Management	The system shall maintain a record of each drone's identity, status and availability.
FR-4	Mission Assignment	The system shall allow an available drone to be assigned to a rescue mission covering a defined zone.
FR-5	Route Planning & Recalculation	The system shall generate a flight route for an assigned drone and recalculate it when an obstacle or new zone is reported.
FR-6	Real-Time Drone Monitoring	The system shall display live telemetry (position, battery, state) for every airborne drone.
FR-7	Sensor / Camera Data Capture	The drone shall continuously capture thermal, visual and environmental sensor readings while executing a mission.
FR-8	Victim & Hazard Detection	The system shall analyse captured data and flag readings that indicate a probable victim or hazardous condition.
FR-9	Alert Generation	The system shall automatically create a prioritised alert whenever a victim or hazard is detected.
FR-10	Notification Dispatch	The system shall push a notification for every alert to the relevant rescue squad and supervising officer.
FR-11	Mission Status Update	The system shall allow the rescue squad and drone pilot to update the status of a mission or an individual alert.
FR-12	Mission Completion Reporting	The system shall generate a structured report summarising activity and outcomes once a mission is closed.

ID	Requirement	Description
FR-13	User & Role Management	The system shall allow a System Administrator to create accounts and assign each user an appropriate role.

6. Non-Functional Requirements

Security: Only authenticated users with an appropriate role may access incident, mission or drone data; role-based access control is enforced at the service layer.

Reliability / Fault tolerance: The loss of a single drone's telemetry link or a temporary service outage shall not corrupt mission state; drones default to a safe return-to-base behaviour on communication loss.

Performance: Sensor-to-alert latency for a high-confidence victim detection shall not exceed a few seconds under normal network conditions.

Scalability: The architecture shall support many concurrent missions and drones without requiring a redesign, by keeping services independently deployable.

Usability: Command-console screens shall present live mission and alert information in a form that a stressed operator can act on within seconds.

Maintainability: Subsystems shall be organised into loosely coupled packages/services so that a change to one (e.g., the detection algorithm) does not require changes to unrelated ones.

Availability: Core mission-management and alerting services shall be deployed for continuous (24×7) operation during active disaster response.

Interoperability: The notification subsystem shall be able to integrate with external SMS/push gateways through a well-defined interface.

7. Use Case Identification

The following use cases were identified from the actor–goal analysis above. Each is traced back to the functional requirement it realises in Section 23 (Traceability).

ID	Use Case	Primary Actor	Traces to
UC-1	Register Disaster Incident	Emergency Control Officer	FR-1
UC-2	Define Affected Zone	Emergency Control Officer	FR-2
UC-3	Assign Drone to Mission	Emergency Control Officer	FR-4

ID	Use Case	Primary Actor	Traces to
UC-3a	Check Drone Availability (included)	—	FR-3
UC-4	Plan Flight Route	Drone Pilot	FR-5
UC-4a	Recalculate Route on Obstacle (extends UC-4)	—	FR-5
UC-5	Monitor Drone Status	Drone Pilot	FR-6
UC-6	Capture Sensor & Camera Data	Autonomous Drone	FR-7
UC-7	Detect Victim / Hazard	Autonomous Drone	FR-8
UC-8	Generate Emergency Alert	(system, includes UC-7)	FR-9
UC-9	Dispatch Notification	Notification Gateway	FR-10
UC-10	Acknowledge Alert	Rescue Squad Leader	FR-11
UC-11	Update Mission Status	Rescue Squad Leader / Drone Pilot	FR-11
UC-12	Generate Mission Report	Emergency Control Officer	FR-12
UC-13	Manage User Accounts	System Administrator	FR-13
UC-14	Handle Drone Fault / Low Battery (extends UC-5)	Autonomous Drone	FR-6

8. Use Case Diagram

Figure 1 shows the ADDRRS system boundary with all six actors and their associated use cases. Two kinds of dependency between use cases are modelled explicitly: an <<include>> relationship is used where one use case always invokes a shared piece of behaviour (for example, generating an alert always includes the detection step that triggered it), while an <<extend>> relationship is used for optional or conditional behaviour that only occurs under certain conditions (for example, a route is recalculated only when an obstacle is encountered, and a fault-handling flow only runs when a fault occurs).

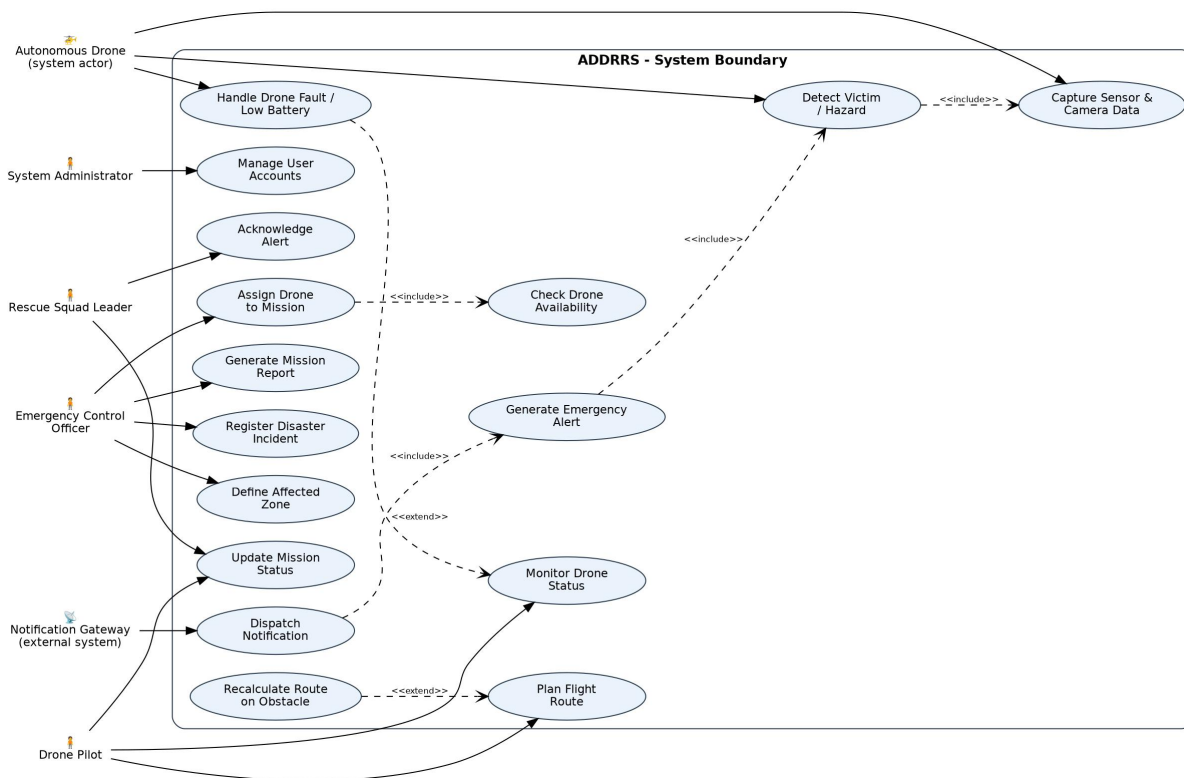


Figure 1 – ADDRRS Use Case Diagram

9. Detailed Use Case Descriptions

UC-3: Assign Drone to Mission

Actor	Emergency Control Officer (primary), Drone Pilot (secondary)
Precondition	An incident and at least one affected zone have already been registered; at least one drone exists in the fleet.
Main Flow	1. The officer selects an affected zone and requests a drone assignment. 2. The system checks drone availability (includes UC-3a) and lists eligible drones. 3. The officer selects a drone and confirms the assignment. 4. The system creates a Mission linking the drone, the zone and the officer, and notifies the assigned Drone Pilot. 5. The Drone Pilot plans a flight route for the mission (UC-4).
Alternate Flow	If no drone is available, the system notifies the officer and the mission request is queued until a drone becomes free.
Postcondition	A Mission object exists in state ASSIGNED with a linked Drone and

	FlightRoute pending execution.
--	--------------------------------

UC-7 / UC-8: Detect Victim or Hazard and Generate Alert

Actor	Autonomous Drone (primary); Rescue Squad Leader and Emergency Control Officer (recipients)
Precondition	A drone is airborne on an active mission and streaming sensor/camera data.
Main Flow	1. The drone continuously captures thermal, visual and gas-sensor readings (UC-6). 2. The detection logic evaluates each reading against configured thresholds. 3. When a reading indicates a probable victim or hazard, a VictimRecord is created and the detection use case includes UC-8 to raise an Alert. 4. The Alert is assigned a priority and the notification use case (UC-9) is triggered automatically. 5. The nearest Rescue Squad Leader and the supervising officer receive a push notification.
Alternate Flow	If the confidence score of a detection is below the alerting threshold, the reading is logged for later review but no alert is raised.
Postcondition	An Alert exists in state RAISED and is visible on the command console until acknowledged (UC-10).

10. Object and Class Identification

Classes were identified using noun-phrase analysis of the problem statement combined with the Boundary–Control–Entity (BCE) classification technique: nouns that a user directly sees or edits became boundary classes, verbs that coordinate a use case became control classes, and the persistent domain nouns that outlive a single use case became entity classes. This keeps user-interface concerns, workflow logic and stored domain data in three separate, independently changeable layers.

Class	Stereotype	Representative Attributes	Responsibility
CommandConsoleUI	Boundary	Screen fields for incident/zone entry, mission dashboard	Presents mission and alert data to the ECO/Admin.
MobileAppUI	Boundary	Alert list, acknowledgement button	Lets a Rescue Squad Leader view and act on alerts in the field.
MissionController	Control	—	Coordinates mission creation, drone assignment and status transitions.
DetectionEngine	Control	confidenceThreshold	Evaluates sensor readings and raises VictimRecord/Alert objects.
AlertManager	Control	—	Creates alerts, tracks priority/escalation, notifies subscribers.
AuthenticationManager	Control	—	Validates credentials and enforces role-based access.
DisasterIncident	Entity	incidentId, type, severity	Represents a registered disaster event.
AffectedZone	Entity	zoneId, boundaryCoords, riskLevel	Represents a geographic area needing survey.
Drone	Entity	droneId, batteryLevel, state	Represents a physical autonomous drone and its live state.
FlightRoute / Waypoint	Entity	routeId / latitude, longitude	Represents the planned path a drone must fly.
Mission	Entity	missionId, status	Links a zone, a drone and a route into a trackable unit of work.

Class	Stereotype	Representative Attributes	Responsibility
SensorReading / CameraFeed	Entity	thermalValue, gasLevel / resolution	Raw data captured by a drone during a mission.
VictimRecord	Entity	victimId, confidenceScore	A candidate victim/hazard derived from sensor data.
Alert / Notification	Entity	priority / channel	A raised event and its delivery record.
RescueTeam / MissionReport / Resource	Entity	teamId / summary / quantity	Supporting domain records used for coordination and reporting.
User (+ four role subclasses)	Entity	userId, role	Represents any authenticated actor and their permissions.

11. Class Diagram

Figure 2 focuses on the entity layer, since this is where the richest structural relationships live. Four kinds of relationship are used: generalisation (the hollow-triangle arrows from the four role classes up to the abstract User class, since every role IS-A user with a login); composition (filled diamonds, used where the part cannot exist without the whole — a Waypoint has no meaning outside its FlightRoute, and a SensorReading is owned by the Drone that captured it); aggregation (the hollow diamond between AffectedZone and Mission, since a zone can exist and be described before any mission ever covers it); and plain association for collaboration that does not imply ownership (for example, a Mission is associated with, but does not own, the Drone assigned to it). A single dependency (dashed arrow) is shown from MissionReport to Mission, since the report reads mission data at generation time but keeps no permanent reference to it.

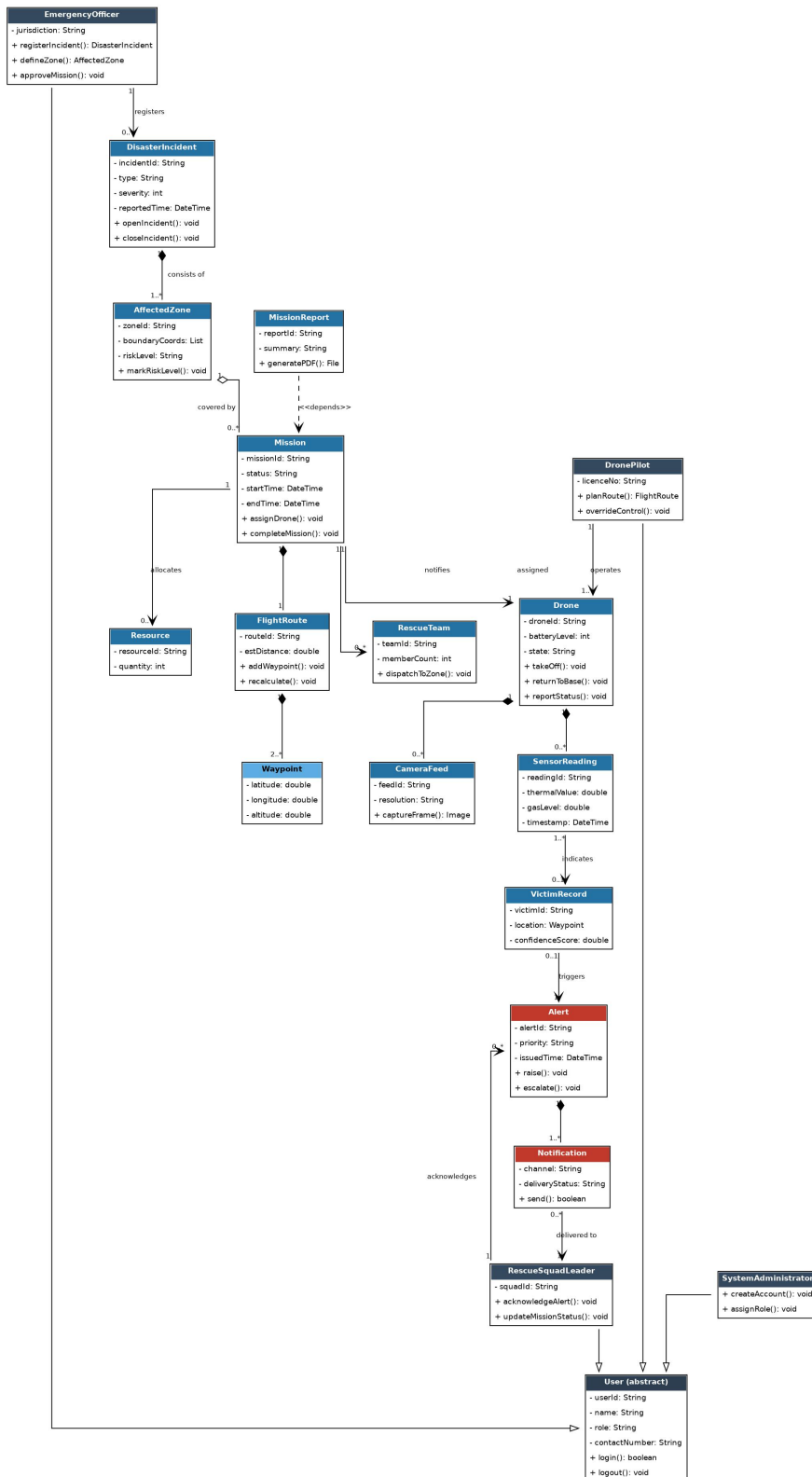


Figure 2 – ADDRRS Class Diagram

12. Sequence / Interaction Diagram

Figure 3 traces the most safety-critical interaction in the system — a drone detecting a possible victim and the resulting alert being pushed to the field. The Sensing/Detection Service performs local

analysis before ever contacting Mission Management, so that noisy or low-confidence readings never leave the edge component. Once a detection is confirmed, Mission Management validates it against the active zone (to suppress detections that fall outside the authorised survey area) before asking the Alert & Notification Service to raise a prioritised Alert. Two independent notification pushes are shown — to the Rescue Squad Leader's mobile app and to the Emergency Control Officer's console — reflecting the Observer-style fan-out used in the design (see Section 20). Acknowledgement flows back asynchronously and, in this scenario, results in the drone being redirected to refine its scan of the immediate area.

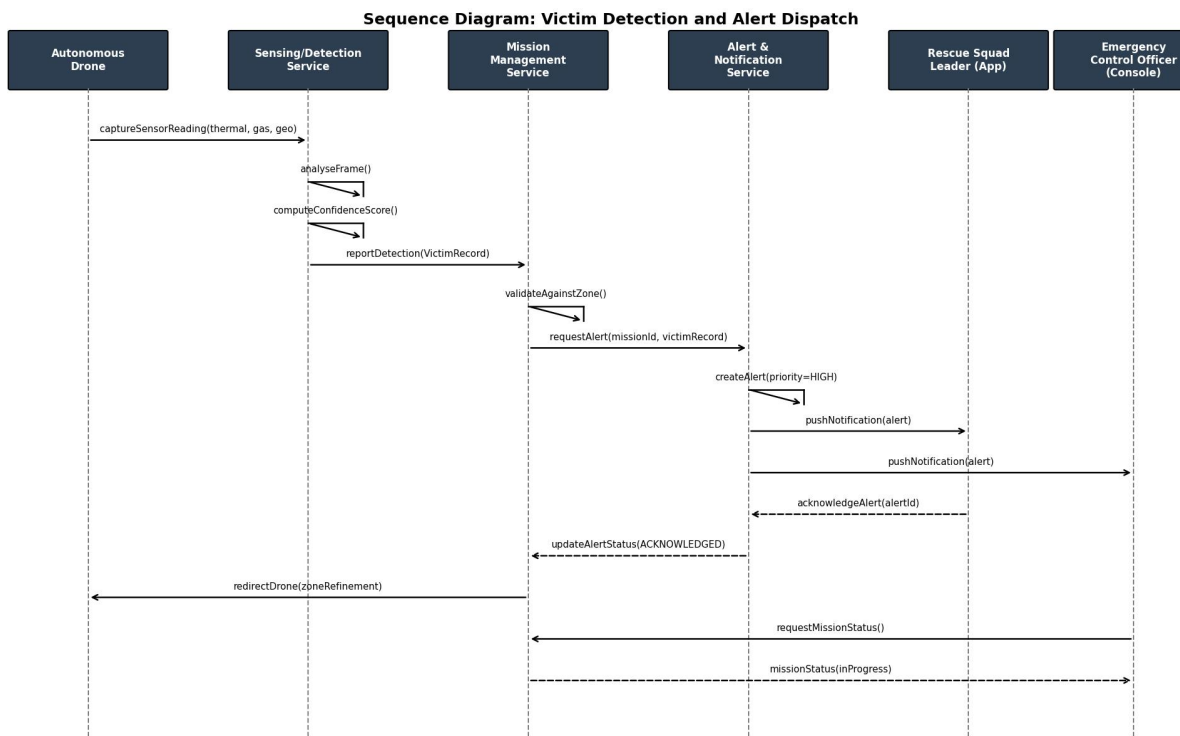


Figure 3 – Sequence Diagram: Victim Detection and Alert Dispatch

13. Activity Diagram

Figure 4 models the end-to-end mission workflow as a single activity diagram, from incident registration through to report generation. Two loops are significant: the drone-availability loop (top), which keeps the officer informed if no drone can be assigned immediately rather than failing the request outright, and the scan/detect loop (lower half), which repeats data capture and detection until either the zone has been fully surveyed or the drone's battery forces an early return. Both loops converge on the return-to-base activity, so that a mission always reaches a well-defined completion and reporting step regardless of which path it took.

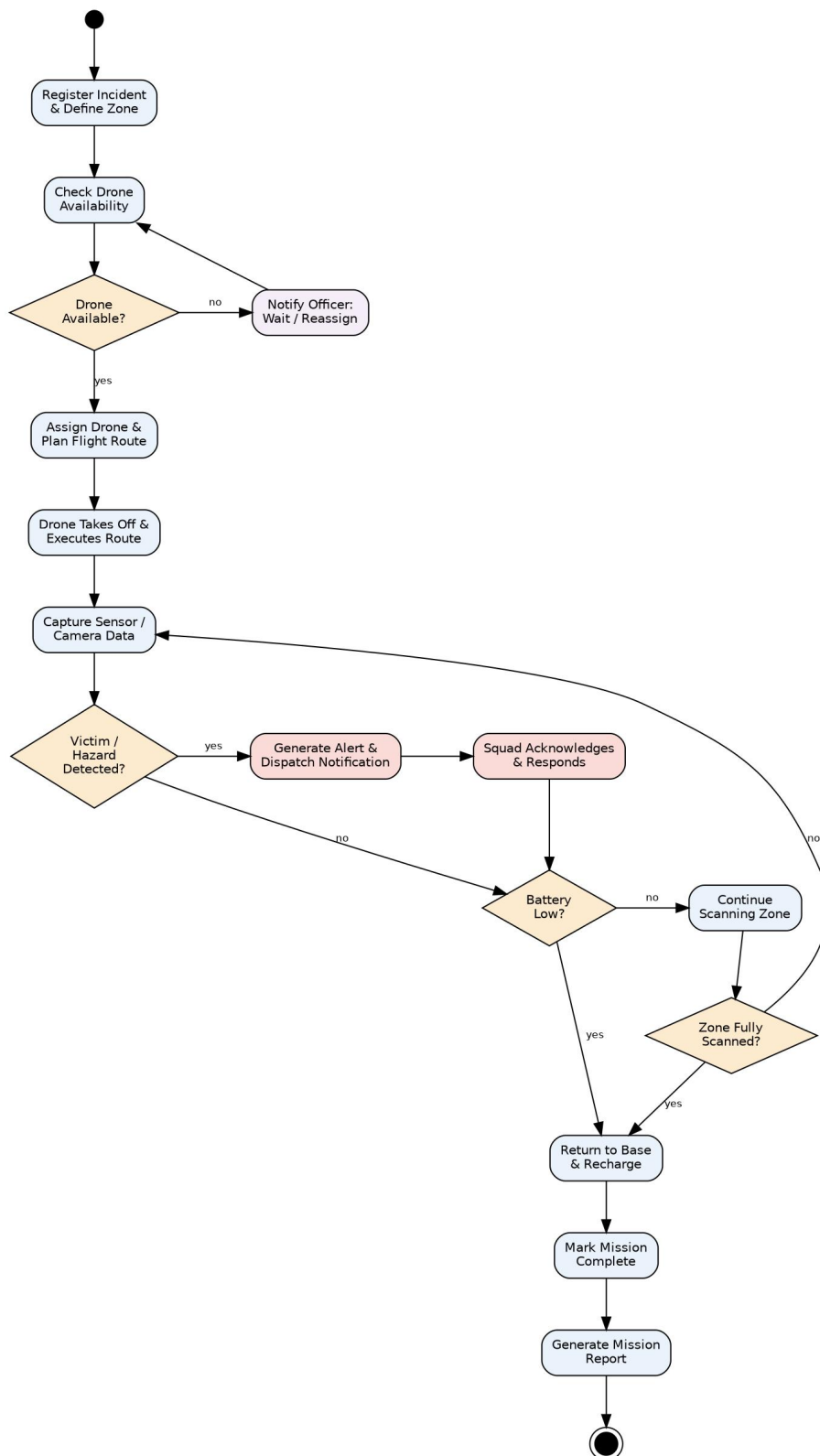


Figure 4 – Activity Diagram: Mission Execution Workflow

14. State Transition Diagram

Figure 5 models the lifecycle of a single Drone object, which is the most behaviourally complex class in the system. The drone begins Idle, moves through Assigned and EnRoute once a mission

starts, and enters Scanning on reaching its zone. A detection event moves it briefly into Alerting before it resumes scanning. A low-battery condition can pre-empt any in-flight state and force a transition to ReturningToBase, after which the drone charges and returns to Idle, ready for reassignment. A faultDetected() event is modelled from every active state into a dedicated Fault state, so that hardware or communication failures are handled uniformly regardless of what the drone happened to be doing when the fault occurred; Maintenance either returns the drone to service or ends its lifecycle through decommissioning.

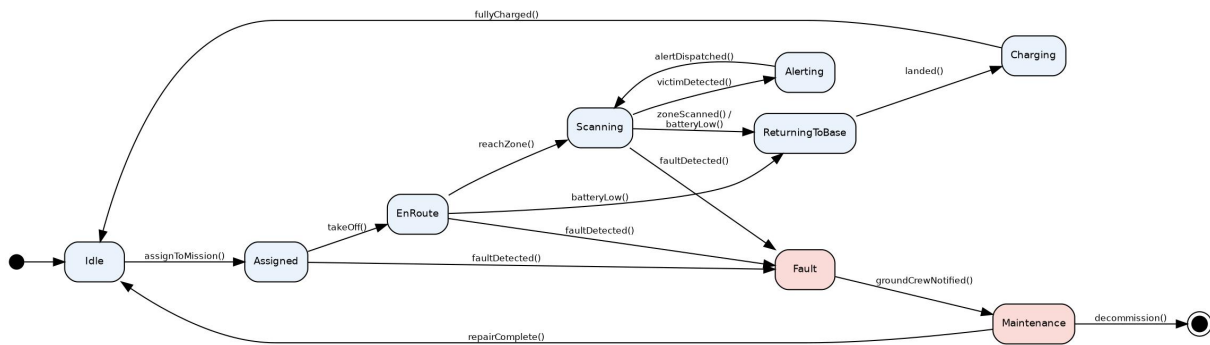


Figure 5 – State Transition Diagram: Drone Lifecycle

15. Package Diagram

Figure 6 groups the identified classes into nine packages, arranged so that dependencies point in one broadly consistent direction (towards Data Persistence) and never form a cycle. The Presentation package depends on the workflow packages (User & Security, Incident & Zone Management, Mission Management) but never the reverse, which keeps the user interface swappable without touching business logic. Mission Management is deliberately the busiest package, since it is the natural coordinator between zone data, drone control and reporting, while Sensing & Detection and Alert & Notification are kept as separate packages from Drone Control so that the detection algorithm can be changed or scaled independently of flight-control logic.

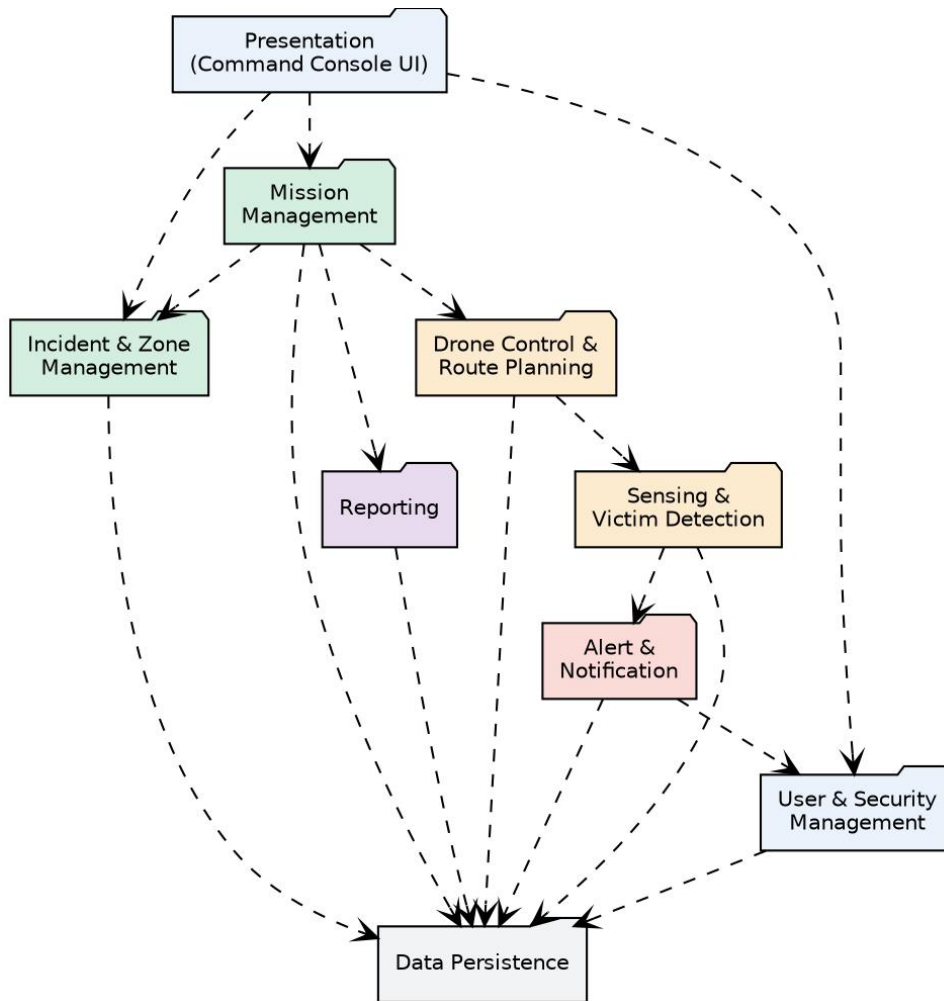


Figure 6 – Package Diagram

16. Component Diagram

Figure 7 refines the package view into deployable components with explicit provided interfaces (the lollipop notation). The Command Console depends only on published interfaces — `IMissionAPI`, `IAuthAPI`, `IReportAPI` and `IAlertPush` — never on the internal implementation of any service, which allows a service to be re-implemented or re-deployed without changing the console. Telemetry flows from the Drone Telemetry & Control Service through a Message Broker rather than through a direct call, which decouples the rate at which drones publish data from the rate at which the Sensor/Video Processing Service consumes it — an important property when many drones may be transmitting simultaneously.

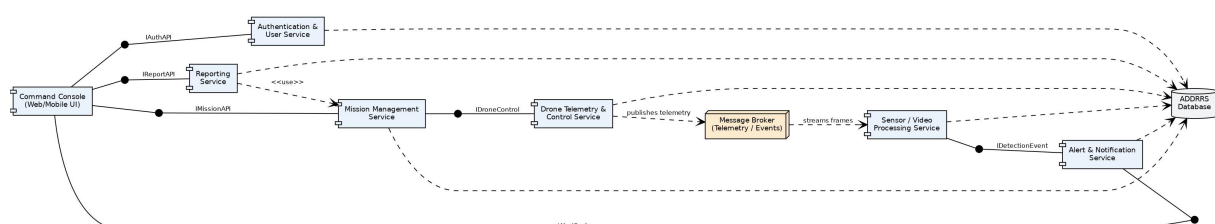


Figure 7 – Component Diagram

17. Deployment Diagram

Figure 8 maps the components onto physical/execution nodes. Each drone carries its own Flight Controller Firmware and Sensor/Camera Module and communicates with the Command Cloud Server over a cellular radio link running a lightweight telemetry protocol. The Ground Control Station (the pilot's laptop) and the Admin Workstation both reach the cloud server over HTTPS/VPN, while the Rescue Squad's mobile app receives alerts through both a direct HTTPS push and, as a fallback, an SMS/push gateway operated by an external provider — giving the notification path redundancy in areas with unreliable data connectivity.

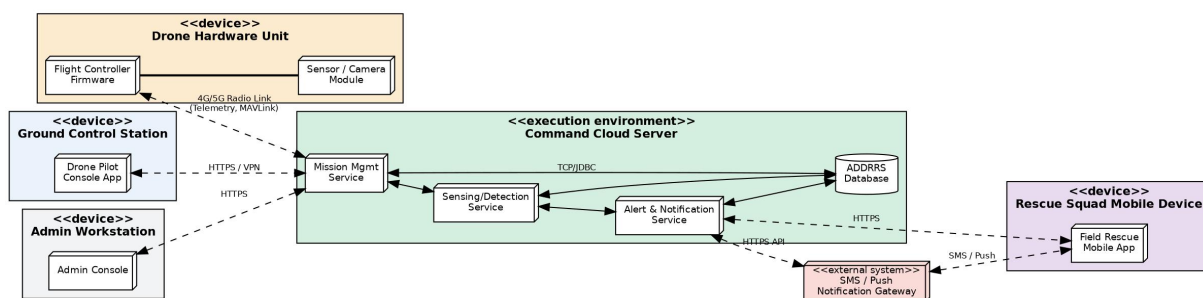


Figure 8 – Deployment Diagram

18. Object Responsibility Analysis

The table below records, in CRC-card style, the primary responsibility of each significant class and the other objects it must collaborate with to fulfil that responsibility. Keeping this list short and specific for each class is itself a design check: a class whose responsibility cannot be stated in one sentence is usually a sign that it should be split.

Class	Responsibility	Collaborators
Mission	Own the lifecycle of a rescue mission; assign a drone; know its route, zone and status	Drone, FlightRoute, AffectedZone, RescueTeam
Drone	Track its own battery/state; execute a route; report telemetry; trigger low-battery return	FlightRoute, SensorReading, CameraFeed
DetectionEngine	Evaluate sensor readings; decide whether a reading warrants a VictimRecord	SensorReading, VictimRecord

Class	Responsibility	Collaborators
AlertManager	Create and escalate alerts; notify subscribed observers	Alert, Notification, RescueSquadLeader, EmergencyOfficer
MissionController	Validate mission requests; enforce zone/availability rules before assignment	Mission, Drone, AffectedZone
AuthenticationManager	Verify credentials; enforce role-based permissions on every request	User and its subclasses
MissionReport	Summarise a completed mission's activity for the officer	Mission, Alert, SensorReading
RescueSquadLeader	Acknowledge alerts; update field status; coordinate the ground response	Alert, Mission

19. Design Axioms and Principles

19.1 Independence Axiom

Suh's Independence Axiom states that a good design maintains the independence of its functional requirements. In ADDRRS this is applied by mapping each major functional requirement onto a distinct package/service (Section 15) rather than letting several requirements share one class: mission coordination, route planning, detection and notification are each satisfied by a separate control class, so a change driven by one requirement — for example, tightening the detection threshold — does not force a change anywhere else.

19.2 Information Axiom

Where more than one design satisfies the Independence Axiom equally well, the Information Axiom favours the option with the least complexity. This is why, for example, notification delivery is modelled as a single Notification entity with a channel attribute rather than as separate SMSNotification and PushNotification classes — the simpler model carries the same information with less structural complexity.

19.3 Encapsulation

Every entity class exposes its state only through methods (e.g., Drone.reportStatus(), Mission.completeMission()); no class in the design reaches into another object's attributes directly, which keeps internal representation free to change.

19.4 Abstraction

The abstract User class captures exactly the behaviour common to every role (login/logout, identity) while leaving role-specific behaviour to its four subclasses, so callers that only need "a user" never need to know which concrete role they are holding.

19.5 High Cohesion

Each class groups only the attributes and methods that serve a single purpose — Drone, for instance, deals only with the drone's own operational state, while all mission-coordination logic lives in Mission and MissionController instead of being scattered across Drone.

19.6 Low Coupling

Cross-package communication happens through a small number of well-named interfaces (Section 16) and, for the highest-volume link (drone telemetry), through a message broker rather than a direct dependency, so the Drone Control and Sensing packages can evolve independently.

19.7 Single Responsibility

AlertManager only manages alerts, DetectionEngine only evaluates readings, and AuthenticationManager only authenticates — none of these control classes also persists data or renders UI, which keeps each one easy to test and reason about in isolation.

20. Design Patterns

Observer — An Alert publishes itself to a list of subscribed AlertObservers (RescueSquadLeader, EmergencyControlOfficer) rather than the AlertManager hard-coding who to notify. This lets the set of interested parties for a given mission change at run time without modifying AlertManager, and is the pattern demonstrated in the sequence diagram's fan-out notification step.

State — The Drone's behaviour (Section 14) is implemented as a family of DroneState classes (Idle, EnRoute, Scanning, ReturningToBase, Fault, ...) rather than as a block of conditional logic inside Drone. Adding a new state, or changing what happens on a given event, touches only one state class instead of a large switch statement.

Strategy — Route planning is defined behind a RouteStrategy interface with interchangeable implementations (e.g., a shortest-path strategy for a simple search and a grid-sweep strategy for a full-area survey). The Drone Pilot can select the strategy appropriate to the mission without MissionController needing to know the planning algorithm's details.

Singleton — The AuthenticationManager and the Message Broker connection are each maintained as a single shared instance per deployment, since having two independent copies of either would risk inconsistent session state or duplicated telemetry delivery.

Factory Method — Creation of the correct User subclass (EmergencyOfficer, DronePilot, RescueSquadLeader, SystemAdministrator) at account-creation time is delegated to a factory method on the User Management control class, keeping the SystemAdministrator's registration use case decoupled from the concrete role classes.

Facade — The Command Console interacts with the whole backend through a single Mission façade interface (IMissionAPI) rather than calling MissionController, DetectionEngine and AlertManager individually, simplifying the client and hiding subsystem reorganisation from the UI.

21. Assumptions

- Each drone is equipped with a thermal camera, a standard camera and a basic gas sensor sufficient to produce the readings the detection logic evaluates.
- Drones maintain a cellular or equivalent long-range data link for most of a mission; brief link loss is tolerated by autonomous safe-return behaviour.
- A drone is assigned to exactly one active mission at a time.
- Victim/hazard detection uses a confidence score derived from sensor fusion; the underlying classification model itself is outside the scope of this design.
- All human actors authenticate through the User/role hierarchy before performing any restricted use case.
- The Notification Gateway is a reliable third-party service reachable over HTTPS from the Alert & Notification Service.
- Zones are represented as simple polygon boundaries sufficient for route planning; detailed terrain modelling is out of scope.
- The system operates within a single command centre per deployment; multi-region federation is not addressed in this design.

22. Constraints

- All UML diagrams must remain consistent with the use cases and classes identified in Sections 7–10.
- Only authenticated, role-appropriate users may invoke any use case that changes system state.

- Sensor-to-alert latency is constrained to a few seconds to remain useful for time-critical rescue decisions.
- Drone autonomous behaviour must include a fail-safe (return-to-base) for both low battery and communication loss.
- The design must not introduce circular dependencies between packages.
- Data persistence is centralised, so every service-level component depends on, but does not duplicate, the shared database.
- The notification channel must degrade gracefully (SMS fallback) when push connectivity is unavailable.

23. Requirement-to-Use Case / Class Traceability

Every functional requirement identified in Section 5 is traceable to at least one use case and to the class(es) that primarily realise it, giving an end-to-end line of sight from requirement to design.

FR ID	Requirement	Use Case(s)	Primary Class(es)
FR-1	Incident Registration	UC-1	DisasterIncident, EmergencyOfficer
FR-2	Zone Definition	UC-2	AffectedZone
FR-3	Drone Fleet Management	UC-3a	Drone
FR-4	Mission Assignment	UC-3	Mission, MissionController
FR-5	Route Planning & Recalculation	UC-4, UC-4a	FlightRoute, Waypoint, DronePilot
FR-6	Real-Time Drone Monitoring	UC-5, UC-14	Drone, Mission
FR-7	Sensor / Camera Data Capture	UC-6	SensorReading, CameraFeed
FR-8	Victim & Hazard Detection	UC-7	VictimRecord, DetectionEngine
FR-9	Alert Generation	UC-8	Alert, AlertManager
FR-10	Notification Dispatch	UC-9	Notification
FR-11	Mission Status Update	UC-10, UC-11	RescueSquadLeader, Mission
FR-12	Mission Completion	UC-12	MissionReport

FR ID	Requirement	Use Case(s)	Primary Class(es)
	Reporting		
FR-13	User & Role Management	UC-13	User, SystemAdministrator

24. Implementation / Prototype and Results

To validate that the class design and the three chosen behavioural patterns (State, Observer, Strategy) actually cohere as an executable model, a small Python console prototype was built independently of the diagrams above (i.e., written from the design, not the other way round). It implements the Drone state machine, an Alert/Observer notification chain, and two interchangeable RouteStrategy implementations, and then runs a single simulated mission end-to-end.

24.1 Prototype Source (excerpt)

```

"""
ADDRRS - Minimal OOAD Prototype
Demonstrates: State pattern (Drone lifecycle), Observer pattern (Alert -> Subscribers),
Strategy pattern (Route planning), and core class collaboration.
This is a conceptual prototype for academic demonstration, not production code.
"""
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
from typing import List

# ----- Strategy Pattern: interchangeable route planning algorithms -----
class RouteStrategy(ABC):
    @abstractmethod
    def plan(self, zone: "AffectedZone") -> List[tuple]:
        ...

class ShortestPathStrategy(RouteStrategy):
    def plan(self, zone):
        return [(zone.center[0], zone.center[1]), (zone.center[0]+0.01, zone.center[1]+0.01)]

class GridSweepStrategy(RouteStrategy):
    def plan(self, zone):
        pts = []
        for i in range(3):
            pts.append((zone.center[0] + i*0.005, zone.center[1]))
        return pts

@dataclass
class AffectedZone:
    zone_id: str
    center: tuple
    risk_level: str = "MEDIUM"

# ----- Observer Pattern: Alert publishes to interested subscribers -----
class AlertObserver(ABC):
    @abstractmethod
    def on_alert(self, alert: "Alert"):
        ...

class RescueSquadLeader(AlertObserver):
    def __init__(self, name):
    ... (full listing continues with Mission, run_demo(), and the remaining pattern classes) ...

```

24.2 Captured Output

```
=====
ADDRRS PROTOTYPE RUN - Mission: Riverside Flood Zone Scan
=====
[Mission M-2031] Route planned with GridSweepStrategy: [(13.0827, 80.2707), (13.0877, 80.2707), (13.0927, 80.2707)]

-- Executing mission --
DR-07: Idle -> EnRoute (takeOff)
Sensor readings processed: 3; possible victim signatures: 1

-- Alert dispatch --
[Alert] Raising HIGH priority alert AL-551 at (13.0827, 80.2707)
[RescueSquadLeader:Squad-Alpha] received alert AL-551 (priority=HIGH) -> dispatching squad to (13.0827, 80.2707)
[EmergencyControlOfficer:R. Menon] logged alert AL-551 in mission dashboard

-- Drone state transitions --
DR-07: EnRoute -> ReturningToBase (batteryLow)
DR-07: ReturningToBase -> Idle (landed & charged)
DR-07: Idle -> EnRoute (takeOff)

=====
Mission M-2031 status: IN_PROGRESS; Drone final state: EnRouteState
=====
```

24.3 Discussion of Results

The captured run confirms three design decisions at once. First, the GridSweepStrategy produces a distinct route shape from what a ShortestPathStrategy would have produced for the same zone, without Mission or MissionController needing any special-case code — evidence that the Strategy pattern is genuinely interchangeable. Second, one of the three simulated sensor readings crosses the victim-detection threshold, and the resulting Alert reaches both subscribed observers (the Rescue Squad Leader and the Emergency Control Officer) through the same raise_alert() call, which is the fan-out behaviour shown in Figure 3. Third, the drone's battery-low and landing events drive it through EnRoute → ReturningToBase → Idle → EnRoute exactly as specified in the state diagram (Figure 5), including the safe rejection of a direct Idle→land() call and a mid-route land() request being redirected to ReturningToBase — showing that illegal transitions are handled by the state objects themselves rather than by ad-hoc checks scattered through the code.

25. SDG Relevance

SDG 3 – Good Health and Well-being — By cutting the time between a disaster occurring and a victim being located, ADDRRS directly targets the survival window that determines outcomes in search-and-rescue operations, and reduces the number of responders who must enter unassessed, hazardous terrain.

SDG 9 – Industry, Innovation and Infrastructure — The system is an applied example of resilient digital infrastructure — autonomous devices, real-time data services and a notification backbone — built specifically to strengthen a community's disaster-response capability.

SDG 11 – Sustainable Cities and Communities — Faster, better-informed rescue coordination is a direct contributor to disaster-resilient urban and rural communities, one of the explicit targets under SDG 11.

SDG 13 – Climate Action — Many of the disaster types ADDRRS is designed for (floods, landslides, wildfires) are increasing in frequency with climate change; equipping emergency services with better situational-awareness tools is part of climate-adaptation, not just disaster response.

26. Reflection on Design Decisions and Learning Outcomes

Working through this assignment made clear how much of OOAD's value comes from the discipline of keeping every diagram consistent with every other diagram, rather than from any single diagram in isolation. The class diagram only became trustworthy once the sequence diagram had been walked through message by message and a missing collaboration (Mission needing to validate a detection against its own zone) surfaced and had to be added back into both the class model and the use case description. That cross-checking loop was the most useful part of the exercise.

Choosing design patterns also turned out to be less about "which pattern is impressive" and more about which one removes a concrete piece of conditional logic. The State pattern was adopted only after sketching the Drone's behaviour as a set of if/elif branches and noticing that the branching was going to grow every time a new state was added; converting it into a small class hierarchy made that growth linear and localised instead of tangled. The Observer pattern came from a similar realisation while drawing the sequence diagram — the Alert-raising step needed to reach two different kinds of recipient, and hard-coding both into AlertManager would have made a third recipient type expensive to add later.

The main challenge was resisting the temptation to over-model: an early draft of the class diagram had almost thirty classes, several of which existed only because a noun appeared once in the problem statement. Applying the Independence and Information axioms from Section 19 as an explicit filter — could two of these classes be merged without losing independence of any functional requirement? — brought the model back down to a set of classes that each earn their place. That exercise, more than any individual UML notation, is the main transferable skill this assignment reinforced.

27. Conclusion

This assignment has produced a complete Object-Oriented Analysis and Design for the Autonomous Drone-Based Disaster Response and Rescue System, spanning requirements analysis, actor and use-case modelling, structural design (class, package, component and deployment diagrams), behavioural design (sequence, activity and state diagrams), and an explicit justification of the design axioms and patterns applied. A working prototype confirmed that the State, Observer and Strategy patterns chosen for the design compose correctly when executed, giving reasonable confidence that the model is not just diagrammatically consistent but behaviourally sound. The resulting design keeps mission coordination, drone control, detection and notification as independent, loosely coupled concerns, which should allow the system to be extended — additional sensor types, additional responder roles, or additional notification channels — without disturbing the rest of the architecture.

28. References

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed.). Prentice Hall.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Object Management Group. (2017). *OMG Unified Modeling Language (UML), Version 2.5.1*. <https://www.omg.org/spec/UML/>
- Suh, N. P. (2001). *Axiomatic Design: Advances and Applications*. Oxford University Press.
- Bahill, A. T., & Chapman, W. L. (1993). A tutorial on quality function deployment. *Engineering Management Journal*, 5(3), 24–35. (Referenced for the design-axiom discussion.)